

check_prebuild_invoked_executables_tracked

check_prebuild_invoked_executables_tracked.sh

Revision: 1 **Last modified:** 2026-09-26T13:40:00Z

Overview

Gate CM-PREBUILD-INVOKED-EXECUTABLES-TRACKED (pre-build invariant 60, BOB-231). Checks that every executable the pre-build sweep runs is git-tracked, so a fresh clone runs the same gates as the authoring host.

Why this exists

BOB-227 found three of the four files behind one pre-build gate untracked. On the authoring host the gate ran; in every fresh clone it did not exist, and there was no committed baseline to compare round over round. This gate makes that whole class visible for every gate, present and future, instead of for one.

Prerequisites

- git, bash, grep, sed, find, sort.
- The repository root must be a git repository. When constitution/ is a checked-out submodule, paths under it are checked against its own index.

Usage

```
bash scripts/pre_build/
    check_prebuild_invoked_executables_tracked.sh .
CM_TRACKED_VERBOSE=1 bash scripts/pre_build/
    check_prebuild_invoked_executables_tracked.sh .    # list
    every checked target
bash scripts/pre_build/
    check_prebuild_invoked_executables_tracked.sh <repo-root>
    <other-sweep-script>
```

What it enumerates

1. Every `${PROJECT_ROOT}/...`, `${SCRIPT_DIR}/...` and `${CONST_GATES_DIR}/...` literal ending in `.sh`, `.py` or `.bash` on a non-comment line of the sweep. A `${var}` inside the path becomes a glob and every match is checked (this covers the per-anchor propagation gates).
2. One level down: inside each resolved script under `scripts/`, the same kind of literal with the bases `${SCRIPT_DIR}` (that script's own directory) and `${REPO_ROOT} / ${PROJECT_ROOT}` — helper analyzers and libraries.
3. Every `tests/**/test_*.sh`, because invariant 30 discovers and runs all of them. An untracked one is an **advisory** WARN, not a failure: a work-in-progress test on a developer machine is normal.

Exit codes

Code	Meaning
0	Every invoked executable is tracked (advisory WARNs may be printed)
1	At least one invoked executable exists on disk but is not tracked
2	Harness error: bad arguments, not a git repo, sweep missing, or zero targets resolved (a blind extraction, never reported as clean)

Edge cases

- A path referenced but absent on disk is printed as `info:` and not counted: it is not a tracking defect (a missing gate fails on its own).
- A path that appears only in a comment is not an invocation.
- The enumeration is static. A script invoked through a path built at run time from something other than a literal plus `${var}` segments, or through a base variable other than the three named above, is not seen. That boundary is not proven empty for the current sweep; measured 2026-09-26 the extraction resolved 159 targets, including all 82 per-anchor propagation gates on disk.

Related scripts

- `tests/pre_build/test_check_prebuild_invoked_executables_tracked.sh` — golden good and bad fixtures, golden-false fixtures, fail-closed arms and a paired mutation.
- `scripts/pre_build/check_gitignore_swallow.sh` — the sibling guard for first-party files swallowed by `.gitignore`.

Last verified

2026-09-26 — real tree: OK: all 159 executables invoked by the pre-build sweep are git-tracked (0 referenced-but-absent, 3 advisory untracked test suite(s)); test suite 12 passed, 0 failed.